Tracking a Timed Trajectory: Kanayama and Feedback Linearisation

Contents

1. Overview . . . 2
2. The reference robot and its timetable . . . 3
   - 2.1 A robot that drives the timetable exactly . . . 4
   - 2.2 Replaying the timetable's speeds: feedforward alone . . . 5
3. Feedback linearisation: steering a point ahead of the axle . . . 5
   - 3.1 The axle midpoint cannot be steered sideways . . . 5
   - 3.2 A point ahead of the axle can move any way . . . 6
   - 3.3 The gap shrinks 5 percent every cycle . . . 7
   - 3.4 What the offset point leaves loose: the heading . . . 9
4. The error in the robot's own frame . . . 10
   - 4.1 Why the room's axes are the wrong ones . . . 10
   - 4.2 How the error changes as both robots drive . . . 12
5. Kanayama's tracker . . . 13
   - 5.1 The rule, part by part . . . 13
   - 5.2 The run . . . 14
   - 5.3 Why both parts are needed . . . 15
6. Why the gap must close: a Lyapunov function . . . 16
   - 6.1 An energy of the error that can only fall . . . 16
   - 6.2 Kanayama's V and why it falls . . . 16
   - 6.3 How fast: choosing the damping . . . 20
7. Where timed tracking runs . . . 23
8. Summary . . . 25
9. Sources . . . 26
10. Key terms . . . 27

Where this fits:

- **Builds on:** Nonholonomic constraints; Feedforward plus feedback; Path following and trajectory tracking.

1. Overview

Key point: A timed trajectory says where the robot should be at every moment; Kanayama's tracker sends the timetable's own speeds plus feedback on the gap, measured along the robot's own axes, and a Lyapunov function proves that the error cannot run away. Our robot starts 0.224 m off its timetable and is within 1 cm of it after 3.35 s.

The room robot of this course, a differential drive, already gets a plan with a clock. Its **local planner** (G-2911), the planner that turns the next few metres of the route into a plan within the robot's limits, hands over a **trajectory** (G-2886): poses, each with the time the robot should be there. The chapter's car can follow a path with no clock: **pure pursuit** (G-3150), which chases a point on the path ahead, and the **Stanley controller** (G-3160), which steers the front wheels from the front axle's errors, bring it back onto a line. The difference between the two jobs is whether the clock counts: **path following** (G-3130) keeps near a path at the robot's own speed, while **trajectory tracking** (G-3131) also keeps to a timetable. Nothing yet makes the room robot keep to a clock.

Here is the room robot's next timetable, with numbers. Starting from (0, 0) and facing along x, it should drive at 0.3 m/s:

1. 1 m straight along x;
2. a 90° left arc of radius 1 m;
3. 1 m straight along y, reaching (2, 2) at 11.90 s.

The timetable is the chapter's car course (10 m straight, a 90° arc of radius 10 m, 10 m straight) shrunk ten times to fit the room. The robot starts at (−0.2, 0.1), also facing along x: 0.2 m behind and 0.1 m to the left of where the timetable says it should be, 0.224 m off.

The clock matters because a timed plan is checked against the world at its own times: a planner that steers round a person crossing the room checks each planned pose against where the person will be at that pose's time (Paden §IV.B). A robot running late can meet the person where the plan said it would already have passed. Figure 1 plays two simple ways to drive the timetable, and both fail (notebook):

- **Replay the timetable's speeds.** The robot drives the right shape from the wrong place: it stays between 0.215 and 0.224 m off all the way, 0.219 m at 11.90 s. Nothing measures the gap, so nothing closes it.
- **Follow the path and ignore the clock.** A path follower, like the car's trackers, steers onto the line but holds its own 0.3 m/s, so it never makes up the 0.2 m it started behind. At 11.90 s it is 0.205 m behind; it reaches (2, 2) at 12.59 s, 0.69 s late.

Every run in this Note is our own Python model of the robot and its trackers (the notebook `RO-073-tracking-a-timed-trajectory.ipynb`), not the code of a real robot stack; Section 7 reads the real code.

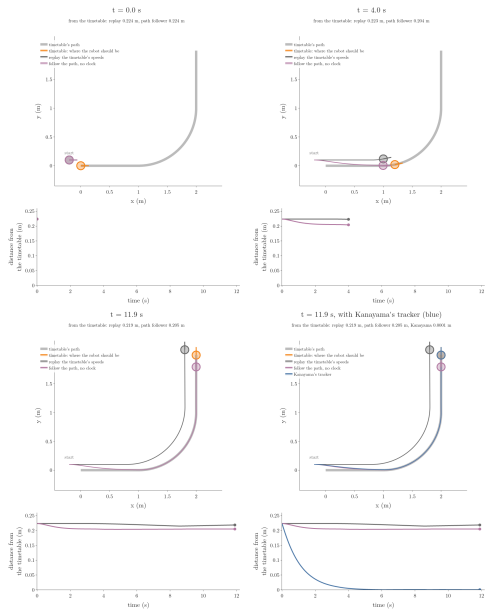


Figure 1: The timetable, one frame per 0.5 s; robots drawn as small circles with a heading line, smaller than their 0.25 m size. The orange robot drives it: where our robot should be at each moment. Grey: our robot replaying the timetable's speeds; purple: a path follower that steers onto the line but ignores the clock. Bottom: each one's distance from where the timetable says it should be. The last frame adds Kanayama's tracker (blue): replay 0.219 m off, path follower 0.205 m behind, Kanayama's tracker 0.0001 m

The last frame of Figure 1 adds the tracker this Note builds (blue): within 1 cm of the timetable from 3.35 s, 0.0001 m at the end. We build it in steps, first steering one point of the robot and then its whole pose:

- a second, imagined robot that drives the timetable exactly, and why replaying its speeds is not enough (Section 2);
- a tracker that makes a point ahead of the axle close its gap by the same fraction every cycle, and the heading it leaves loose (Section 3);
- the whole gap, heading included, measured along our robot's own axes, and how it changes as both robots drive (Section 4);
- Kanayama's tracker, the timetable's speeds plus feedback on that gap (Section 5);
- the proof that the error cannot run away, and how fast the gap closes (Section 6);
- where timed tracking runs in Autoware and Nav2 (Section 7).

2. The reference robot and its timetable

> **Key point:** Picture a second robot that drives the timetable exactly, the reference robot; tracking means staying on top of it. Copying its speeds only works from where it starts: two robots facing the same way with the same commands keep the same gap for ever.

2.1 A robot that drives the timetable exactly

The opening's target moves: where the timetable says to be changes every moment. So we picture a second robot that drives the timetable exactly, always where our robot should be, facing the way it should face, at the speeds it should drive. This imagined robot is the **reference robot** (G-3180), the orange robot of Figure 1 (Kanayama 1991 §2.2 calls its pose the reference configuration). Our robot's job is to drive on top of it, which is **tracking** (G-2954), following a **reference** (G-2944) that changes over time.

The reference robot at time t:

- x_r and y_r are its position, in m: (1.717, 0.303) at $t = 6$ s, on the arc;
- θ_r is its heading, in rad: 45.8° at $t = 6$ s;
- v_r is its forward speed, 0.3 m/s all the way;
- ω_r is its turn rate, in rad/s: 0 on the straights, 0.3 rad/s on the arc.

On the arc it turns at its speed divided by the radius, the **turning radius** (G-2293) rule of RO-001 run backwards:

$$\omega_r = 0.3/1.0 = 0.3 \text{ rad/s}$$

The timetable's three parts take:

$$1/0.3 = 3.333 \text{ s}$$

$$(\pi/2) \times 1/0.3 = 5.236 \text{ s}$$

$$1/0.3 = 3.333 \text{ s}$$

So the arc starts at 3.333 s, ends at 8.569 s, and the timetable ends at 11.903 s, which we write 11.90 s.

Our robot is the differential drive of RO-001: its pose is x, y and θ, and we command its forward speed v and its turn rate ω. The **kinematic model** (G-2297) turns them into motion, the robot moving the way it faces:

$$\dot{x} = v \cos\theta$$

$$\dot{y} = v \sin\theta$$

$$\dot{\theta} = \omega$$

Each wheel turns at most 0.6 m/s, and a tracker sends a new command 20 times a second (20 Hz), the rate of Nav2's sample controller configuration (Nav2 `nav2_params.yaml`). The simplest way to keep to the timetable is to drive what the reference robot drives.

2.2 Replaying the timetable's speeds: feedforward alone

The opening's first failure is this simplest tracker: it sends the reference robot's own speed and turn rate as its commands. A command computed from the plan, before any error appears, is **feedforward** (G-3070). Figure 1 (grey) shows it on our start: it drives the timetable's shape 0.2 m behind and 0.1 m to the left, and the gap never closes.

Why the gap stays. Both robots start facing 0° and turn at the same rate, so at every moment they face the same way. Both drive at 0.3 m/s. The rate of the gap along x is the reference robot's x speed minus ours; at $t = 6$ s, both facing 45.8°:

$$0.3\cos 45.8^\circ - 0.3\cos 45.8^\circ = 0$$

$$v_r \cos\theta_r - v\cos\theta = 0$$

The same holds along y with sines, and the headings turn at the same rate. So the gap of 0.2 m and 0.1 m stays for ever; in the run the 20 Hz command steps shift it only a little, between 0.215 and 0.224 m. Feedforward alone keeps any error the robot starts with or meets on the way (MR §11.3.1), so it works only from a start exactly on the timetable. To close the gap, the tracker must measure it.

3. Feedback linearisation: steering a point ahead of the axle

> **Key point:** A point P a distance d ahead of the axle can move in any direction, because turning swings it sideways at d times the turn rate, so a law can make its gap to the reference shrink by the same fraction every cycle, at any size. The heading is left out of that law: with P behind the axle, the robot turns round and drives the timetable backwards with P on time.

3.1 The axle midpoint cannot be steered sideways

Replaying the speeds left the gap of 0.2 m and 0.1 m in place. To close it, the tracker must measure the gap and steer against it. The most direct way steers our robot's position straight at the reference robot's: at the start, move 0.2 m/s along x and −0.1 m/s along y on top of the timetable's speed. But the position of our robot is the axle midpoint, which can only move along its heading, the **nonholonomic constraint** (G-2302) of RO-001: facing along x, it has no way to move along y.

3.2 A point ahead of the axle can move any way

Take instead a point P fixed to the robot a distance d ahead of the axle midpoint (MR §13.3.1, Figure 13.12; Paden §V.B.2). Figure 2 shows why P can move any way: it moves ahead with the robot at v, and when the robot turns, it swings sideways round the midpoint at d times ω, by the arc rule of RO-001 (speed is turn rate times distance).

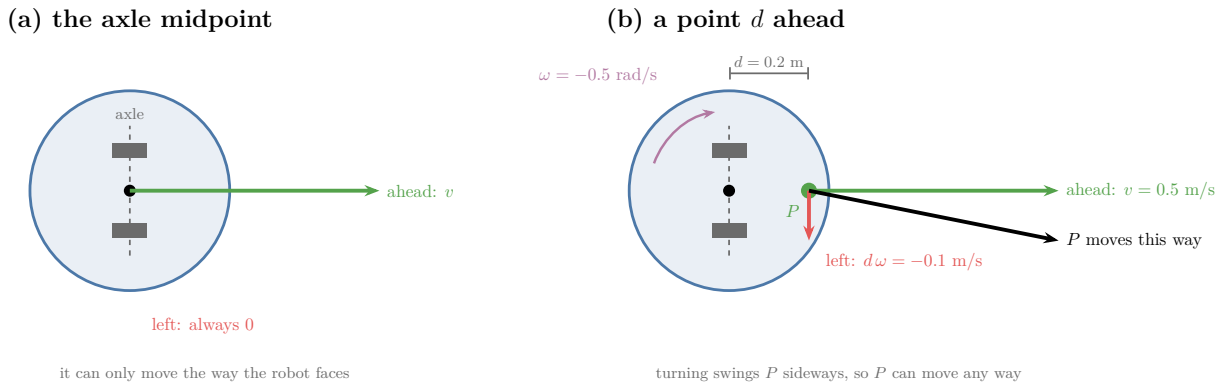


Figure 2: The robot and the speeds drawn to one scale. (a) The axle midpoint moves only ahead, at v; its sideways speed is always 0. (b) A point P a distance d = 0.2 m ahead moves ahead at v = 0.5 m/s and, while the robot turns at $\omega = -0.5$ rad/s, sideways at d $\omega = -0.1$ m/s; together P moves 0.5 m/s ahead and 0.1 m/s to the right

Such a point is an **offset point** (G-3185). The symbols, at the start of our run:

- d is the distance of P ahead of the axle midpoint, 0.2 m, the front of the robot's 0.25 m radius;
- u_a is the speed we want for P along the robot's heading, 0.5 m/s at the start;
- u_l is the speed we want for P to the robot's left, −0.1 m/s at the start;
- v and ω are the commands.

P moves ahead at v and to the left at $d\omega$, so the commands that give the wanted velocity are:

$$v = u_a = 0.5 \text{ m/s}$$

$$\omega = u_l/d = -0.1/0.2$$

$$= -0.5 \text{ rad/s}$$

The wheels then turn at 0.45 and 0.55 m/s, under the limit. At $d = 0$ the second line divides by zero: the midpoint has no sideways speed to give, and the commands for it do not exist

(MR §13.3.1 calls the matrix from v and ω to the point's velocity singular there). A small d asks for large turn rates: the same -0.1 m/s sideways with $d = 0.05$ m would need -2 rad/s.

Why P moves that way. P sits at the midpoint plus d along the heading. In the room's axes, at heading θ:

$$x_P = x + d\cos\theta$$

$$y_P = y + d\sin\theta$$

Taking the rate of each line, with $\dot{\theta} = \omega$ and the rates of the cosine and sine:

$$\dot{x}_P = v\cos\theta - d\,\omega\sin\theta$$

$$\dot{y}_P = v\sin\theta + d\,\omega\cos\theta$$

At the start, $\theta = 0$:

$$\dot{x}_P = 0.5 \times 1 - 0 = 0.5 \text{ m/s}$$

$$\dot{y}_P = 0 + 0.2 \times (-0.5) = -0.1 \text{ m/s}$$

The first terms are the robot driving ahead, the second the swing of P round the midpoint; read along the robot's own axes they are v ahead and $d\,\omega$ to the left (MR eq. 13.20 and 13.21). So any wanted velocity of P can be commanded; which velocity to want is the next step.

3.3 The gap shrinks 5 percent every cycle

Now choose P's wanted velocity as a feedforward part, the reference point's own velocity as in Section 2.2, plus a feedback part on the gap. The reference robot has its own point P_r, 0.2 m ahead of its axle. At the start:

- P_r is at (0.2, 0) m and moves at 0.3 m/s along x, its velocity $\dot{P}_r$;
- P is at (0, 0.1) m, 0.2 m ahead of our robot at $(-0.2, 0.1)$;
- k is the gain, 1 per s;
- u is P's wanted velocity in the room.

$$u = \dot{P}_r + k\,(P_r - P)$$

Along x and along y at the start:

$$u_x = 0.3 + 1 \times (0.2 - 0) = 0.5 \text{ m/s}$$

$$u_y = 0 + 1 \times (0 - 0.1) = -0.1 \text{ m/s}$$

The two speeds are those of Figure 2b (facing along x, the robot's axes are the room's). Section 3.2 turns u into v and ω exactly, so P moves at u. The gap e is the reference point minus ours, an arrow with an x part and a y part, 0.224 m long at the start:

$$e = P_r - P$$

Its rate:

$$\dot{e} = \dot{P}_r - u$$

$$= -k\, e$$

The gap shrinks at k times itself, whatever its size and whatever the robot's heading: an exactly straight-line (**linear** (G-2527)) rule, with no sines or products. Choosing the commands through the inverse of the robot's model so that an output obeys a linear rule is **feedback linearisation** (G-3184) (Paden §V.B.2, output feedback linearization; MR eq. 13.29 and 13.21 steer P the same way without the feedforward part). The law is the feedforward plus proportional feedback that makes an error decay exponentially (MR §11.3.1, eq. 11.15 without its integral part), made possible for a robot that cannot slide sideways by steering a point ahead of the axle.

Our tracker sends one command every 0.05 s, so in each cycle the gap shrinks by k times 0.05, 5 percent. After 20 cycles, 1 s:

$$0.95^{20} = 0.3585$$

$$0.2236 \times 0.3585 = 0.0802 \text{ m}$$

The run gives 0.0800 m (notebook); a gap shrinking smoothly at the same rate, with no 0.05 s steps, would be at 0.0823 m. Figure 3 (left) plays it: the gap of P (bottom, solid green) lies on the grey curve of 5 percent per cycle, 0.010 m at 3 s, and the robot ends on the timetable, its axle 0.0001 m from where it should be. The law steers only P, though; the body comes along by itself.

3.4 What the offset point leaves loose: the heading

The law of Section 3.3 holds P on time but says nothing about which way the body faces. MR §13.3.4 allows P anywhere on the robot off the axle line, so Figure 3 (right) runs the same law with P 0.2 m behind the axle ($d = -0.2$ m). Watch the right-hand robot: within about 3 s it swings round and then drives the whole timetable backwards.

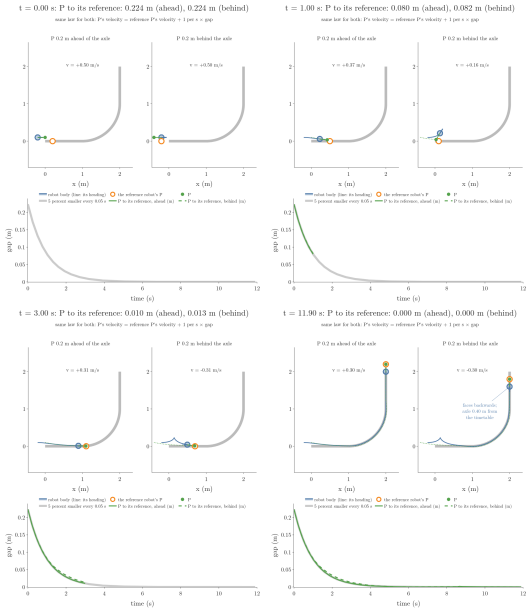


Figure 3: Feedback linearisation on the timetable, one frame per 0.25 s up to 4 s, then one per second; the body drawn as a small circle, not to size. Both panels use the same law with gain 1 per s. Left: P (green dot) 0.2 m ahead of the axle; the body (blue) follows it. Right: P 0.2 m behind the axle; the body swings round, leaving a cusp in its track, and drives backwards. Orange rings: the reference robot's P. Bottom: the gap from P to its reference, ahead (solid) and behind (dashed), against a gap that shrinks 5 percent every 0.05 s (grey). The last frame shows both gaps near 0 and, on the right, the body facing backwards with its axle 0.40 m from the timetable

P's gap shrinks just as fast in both runs: 0.0800 m and 0.0820 m at 1 s. But on the right the robot ends facing 180° the wrong way at -0.30 m/s, and its axle is 0.40 m from where the timetable puts it, twice d (notebook). MR §13.3.4 shows the same failure for its point tracker: the point converges, but "the robot's orientation is opposite to the intended orientation" (MR Figure 13.19).

Why the heading goes where it goes. Once P is on time, P moves with P_r, at 0.3 m/s along the timetable on a straight. If the timetable heads at an angle to the left of our robot, the part of that velocity to our robot's left is 0.3 m/s times the sine of the angle, and Section 3.2 turns the robot at that sideways speed divided by d. The symbols:

- φ is the angle the timetable heads to the left of our robot, 10° in the example, where its sine is 0.1736;

- v_r is the timetable's speed, 0.3 m/s;
- d is the distance of P ahead of the axle, 0.2 m (-0.2 m when P is behind);
- ω is the turn rate the law then commands, in rad/s.

$$\omega = 0.3 \times 0.1736/0.2$$

$$= 0.26 \text{ rad/s}$$

$$\omega = v_r \sin\varphi/d$$

With P ahead, the robot turns left, towards the timetable's heading, so φ shrinks. With $d = -0.2$ m, the same line gives -0.26 rad/s: the robot turns away and φ grows until the sine is zero again at 180°, facing backwards. A trailer behaves the same way: towed with its wheels behind the hitch it follows, and pushed with its wheels in front it swings round. Here P is the hitch and the robot's wheels are the trailer's.

Feedback linearisation buys an exactly linear gap and a single gain, at the price of a loose heading, turn rates that grow as d shrinks, and forward driving with P ahead (Paden Table II: "forward driving only"). To keep the heading, a tracker must measure it, together with the rest of the gap.

4. The error in the robot's own frame

> **Key point:** The robot can only speed up, slow down and turn, so the gap to the reference robot is measured along the robot's own axes: x_e ahead, y_e to the left, and θ_e, the reference heading minus ours. The same gap in the room means "speed up" for a robot facing one way and "turn right" for a robot facing another.

4.1 Why the room's axes are the wrong ones

The offset point measured only P's gap, along the room's axes, and left the heading loose. Kanayama measures the whole gap, heading included, from the robot itself. Take the opening's gap again: in the room, the reference robot is 0.2 m further along x and 0.1 m lower in y. What our robot should do about that depends on which way it faces (Figure 4):

- **Facing 0°, at the start:** the reference robot is 0.2 m straight ahead and 0.1 m to the right, so the robot should speed up and turn a little right.
- **Facing 45°, halfway round the arc while replaying:** the same gap in the room is only 0.071 m ahead but 0.212 m to the right, so the robot should hardly speed up and should turn right.

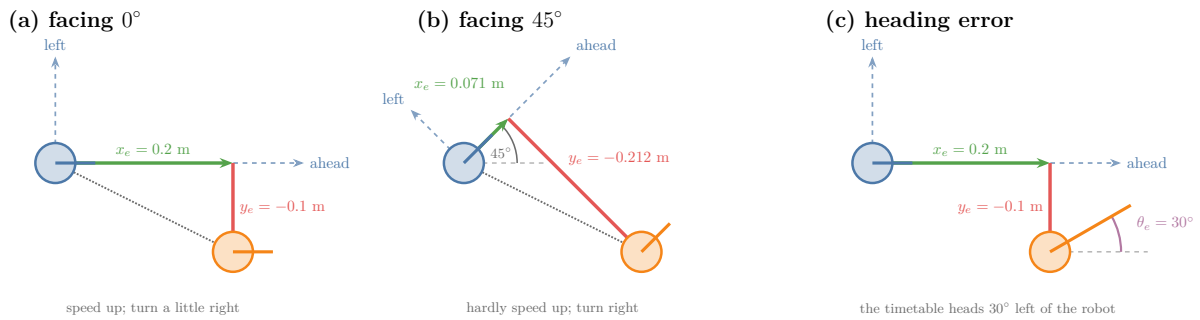


Figure 4: The error in the robot's own frame, drawn 20 cm per metre. Blue: our robot with its own ahead and left axes; orange: the reference robot. (a) Facing 0°, the reference robot 0.2 m along x and 0.1 m down y is x_e = 0.2 m ahead and y_e = −0.1 m (0.1 m to the right). (b) Facing 45°, the same gap in the room is x_e = 0.071 m ahead and y_e = −0.212 m. (c) The third number, θ_e: the reference robot's heading minus ours, here 30°

The robot's two commands act along its own axes: v moves it ahead, ω turns it, and the nonholonomic constraint (G-2302), the rule that its wheels cannot slide, stops it moving sideways. So we measure the gap along those axes (Kanayama 1991 eq. 3):

- x_e is how far the reference robot is ahead of ours, along our heading, in m;
- y_e is how far it is to our left, in m, negative to the right;
- θ_e is its heading minus ours, in rad, positive when it heads further left.

The three numbers together are the **error in the robot's frame** (G-3181). They are close cousins of three errors of RO-070, measured from the path: the **along-track error** (G-3137), how far behind the timetable; the **lateral error** (G-3135), how far to the side of the path; and the **heading error** (G-3136), the heading minus the path's. Kanayama measures from the robot towards the reference robot instead, so the signs are flipped: 0.1 m to the left of the timetable is a lateral error of +0.1 m and a y_e of −0.1 m. The gap in the room, for the formula:

- Δx is the reference robot's x minus ours, 0.2 m in Figure 4;
- Δy is its y minus ours, −0.1 m;
- θ is our heading, 45° in panel (b).

$$x_e = \Delta x \cos\theta + \Delta y \sin\theta$$

$$y_e = -\Delta x \sin\theta + \Delta y \cos\theta$$

$$\theta_e = \theta_r - \theta$$

Facing 45°, where both the cosine and the sine are 0.7071:

$$x_e = 0.2 \times 0.7071 - 0.1 \times 0.7071$$

$$= 0.0707 \text{ m}$$

$$y_e = -0.2 \times 0.7071 - 0.1 \times 0.7071$$

$$= -0.2121 \text{ m}$$

Why it holds. Our robot's ahead axis points along $(\cos\theta, \sin\theta)$ and its left axis along $(-\sin\theta, \cos\theta)$, the split of a speed into cosine and sine parts. Each line takes the part of the room gap that lies along one axis. A check at $\theta = 0$: the cosine is 1 and the sine 0, so the gap ahead is the gap along x and the gap to the left is the gap along y, as panel (a) shows; facing along x, the robot's axes are the room's. The error now says where the reference robot is; what the commands do to the error is still open.

4.2 How the error changes as both robots drive

A tracker needs to know what its commands do to the three errors, x_e ahead, y_e to the left and θ_e in heading. Three motions change them:

1. **The reference robot drives** at v_r along its heading, which is θ_e from ours. So it moves ahead of us at $v_r \cos\theta_e$ and to our left at $v_r \sin\theta_e$, the same cosine and sine split.
2. **Our robot drives** at v along its own heading, so everything ahead comes closer: x_e falls at v.
3. **Our robot turns** at ω. Its axes turn left, so a point standing still in the room swings right as our robot sees it, at ω times its distance, the arc rule of RO-001. A point ahead slides to the right: y_e falls at ωx_e. A point to the left slides ahead: x_e grows at ωy_e. The heading error falls at ω while the reference robot turns it up at ω_r.

Adding the three (Kanayama 1991 Lemma 1; Paden §V.B.1):

$$\dot{x}_e = \omega y_e - v + v_r \cos\theta_e$$

$$\dot{y}_e = -\omega x_e + v_r \sin\theta_e$$

$$\dot{\theta}_e = \omega_r - \omega$$

These rates are the **error dynamics** (G-2986) of tracking: how the error moves for given commands. A check on the opening's start, replaying the timetable, with $x_e = 0.2$ m, $y_e = -0.1$ m, $\theta_e = 0$, $v = 0.3$ m/s and $\omega = 0$:

$$\dot{x}_e = 0 - 0.3 + 0.3 \times 1 = 0$$

$$\dot{y}_e = 0 + 0.3 \times 0 = 0$$

$$\dot{\theta}_e = 0 - 0 = 0$$

All three rates are zero, the frozen gap of Section 2.2. The rates also say what can close it: a speed above $v_r \cos\theta_e$ shrinks x_e, and a turn changes all three. Which commands make all three shrink together is the next question.

5. Kanayama's tracker

> **Key point:** Kanayama's tracker drives the reference robot's speeds and corrects them by the error: it speeds up in proportion to the gap ahead and turns in proportion to the gap to the left and to the sine of the heading gap, both scaled by the reference speed. On our start it is within 1 cm of the timetable after 3.35 s and asks at most 0.548 m/s of a wheel.

5.1 The rule, part by part

From the start, the reference robot is 0.2 m ahead and 0.1 m to the right, facing our way (Figure 4a). The rule of Kanayama 1991 (eq. 8; Paden §V.B.1) has a feedforward part and a feedback part in each command:

- **Feedforward**, $v_r \cos\theta_e$ and ω_r: drive what the timetable drives. The speed is the part of the reference robot's speed that lies along our heading, so by rule 1 of Section 4.2 the reference robot then pulls ahead no faster than we follow. "The first term in each velocity is a feedforward part" (Kanayama 1991 §3).
- **Speed feedback**, K_x times x_e: the reference robot is ahead, so drive faster, in proportion to the gap, **proportional control** (G-2958).
- **Sideways feedback**, $v_r K_y y_e$: the reference robot is to the left, so turn left. The reference speed multiplies it because turning moves a robot sideways only as fast as it drives.
- **Heading feedback**, $v_r K_\theta \sin\theta_e$: turn towards the reference robot's heading.

The symbols, with the values this Note uses (chosen in Section 6.3):

- v_r and ω_r are the reference robot's speed and turn rate, 0.3 m/s and 0 on the first straight;
- x_e, y_e and θ_e are the error in the robot's frame, 0.2 m, −0.1 m and 0 at the start;
- K_x is the speed gain, 1 per s: 1 m/s more speed for each metre of gap ahead;
- K_y is the sideways gain, 16 per m^2;
- K_θ is the heading gain, 8 per m;
- v and ω are the commands, in m/s and rad/s.

$$v = v_r \cos\theta_e + K_x x_e$$

$$\omega = \omega_r + v_r(K_y y_e + K_\theta \sin\theta_e)$$

This rule is **Kanayama's tracker** (G-3182). At the start:

$$v = 0.3 \times 1 + 1 \times 0.2$$

$$= 0.5 \text{ m/s}$$

$$\omega = 0 + 0.3 \times (16 \times (-0.1) + 0)$$

$$= -0.48 \text{ rad/s}$$

The robot speeds up and turns right, towards the reference robot. The wheel speeds for it, with half the wheel separation 0.1 m:

$$v_R = 0.5 + (-0.48) \times 0.1 = 0.452 \text{ m/s}$$

$$v_L = 0.5 - (-0.48) \times 0.1 = 0.548 \text{ m/s}$$

Both are under the 0.6 m/s limit. A check at zero error: the cosine of 0 is 1 and the feedback parts are 0, so the commands are the reference robot's own speed and turn rate, which keep a robot that is on the timetable on it (Section 2.2). The tracker of MR §13.3.4 has the same property: with zero error it sends the planned controls. Section 5.2 runs the rule from the opening's start.

5.2 The run

Figure 5 plays the rule from the opening's start, 0.224 m off. The right panel shows each moment as our robot sees it, the robot at the centre facing up: watch the green (x_e) and red (y_e) bars shrink as the orange reference robot comes to sit on top of ours.

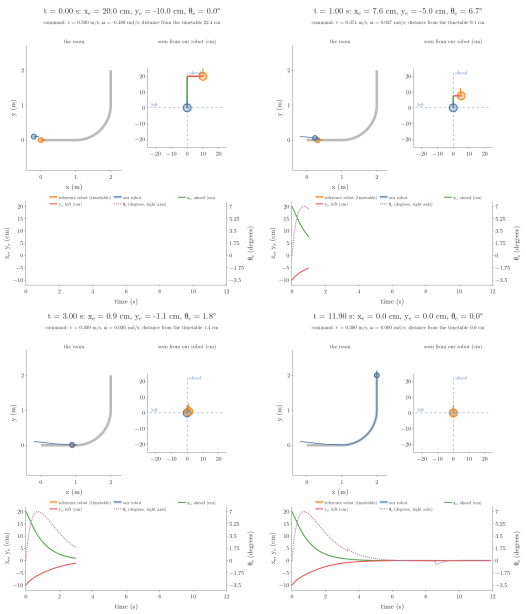


Figure 5: Kanayama's tracker on the timetable, one frame per 0.25 s up to 4 s, then one per second; robots drawn as small circles, not to size. Left: the room, the reference robot (orange) and our robot (blue). Right: the same moment seen from our robot, which sits at the centre facing up; the reference robot is x_e ahead (green bar) and y_e to the left (red bar, to the right when negative), and its heading line leans by θ_e. Bottom: x_e and y_e in cm, θ_e in degrees on the right axis. The first frame shows x_e = 20 cm, y_e = −10 cm and the command v = 0.500 m/s, $\omega = -0.480$ rad/s; the last frame shows the robot on the timetable at 11.90 s

The distance from the timetable falls from 22.4 cm at the start to 9.1 cm at 1 s and 1.4 cm at 3 s, and stays under 1 cm from 3.35 s, just as the arc begins. On the arc, the feedforward ω_r does the turning, and at 11.90 s the robot is 0.01 cm from where it should be (notebook). The heading error rises to 7.0° in the first second, while the robot turns towards the line, and then falls back; the largest wheel speed is the 0.548 m/s of the first command. Feedback closed the gap; whether the feedforward parts are still needed is the next check.

5.3 Why both parts are needed

Drop the two feedforward parts, so the rule is:

$$v = K_x x_e$$

$$\omega = v_r(K_y y_e + K_\theta \sin\theta_e)$$

Then the robot never catches up: it is 0.287 m behind at 2 s and 0.297 m behind at the end, about 1 s late, and on the arc its heading lags by up to 15° (notebook). The speed now has to come from the gap. To drive at 0.3 m/s with a speed gain of 1 per s, the gap ahead must be:

$$x_e = 0.3/1 = 0.3 \text{ m}$$

$$x_e = v_r/K_x$$

The 0.3 m is the same kind of lag as RO-066's speed loop, which ended 10 cm behind its plan because its feedback also had to fall behind before it pushed. The two parts share the work: feedforward drives the timetable, and feedback removes the gap the start left. One run converged, but one run is not a guarantee; the next section asks whether every start does.

6. Why the gap must close: a Lyapunov function

> **Key point:** Kanayama's V, half the squared distance to the reference robot plus a heading part, is zero only on the timetable and positive everywhere else, and under his tracker its rate is never above zero. So V can never grow, from any start, as long as the reference robot drives forward, and the error cannot run away; the gap closes fastest without swinging past the line when the damping ratio is 1.

6.1 An energy of the error that can only fall

Figure 5 showed one start closing its gap. To know that every start does, without trying them all, we use a picture from everyday life. A marble rolling inside a bowl with some friction loses height all the time, so it must end at the bottom, wherever it started.

So we look for one number built from the error that behaves like the marble's height:

1. it is zero when the error is zero, and positive for every other error;
2. under the tracker it never rises.

Such a number is a **Lyapunov function** (G-3183) (Kanayama 1991 §3; Paden §V.B.1). If one exists, the number can never rise above its start value, so the error cannot run away; if it also keeps falling until the error is zero, the robot ends on its timetable. Kanayama turned to one because the robot's equations have sines and products and only two commands for three errors, and "the use of a Liapunov function resolves these difficulties" (Kanayama 1991 §1): the proof below needs only the error rates of Section 4.2, never their solution.

6.2 Kanayama's V and why it falls

Kanayama's choice adds a distance part and a heading part:

- V is the Lyapunov function, in m^2;
- x_e and y_e are the gap ahead and to the left, in m (0.2 and -0.1 at the start);
- θ_e is the heading gap, in rad (0 at the start);
- K_y is the sideways gain, 16 per m^2.

$$V = \frac{x_e^2 + y_e^2}{2} + \frac{1 - \cos\theta_e}{K_y}$$

At the start:

$$V = \frac{0.04 + 0.01}{2} + 0$$

$$= 0.025 \text{ m}^2$$

V meets the first condition: the distance part is half the squared distance to the reference robot, and the heading part is zero at $\theta_e = 0$ and positive at every other heading gap, because a cosine is below 1 there. Dividing it by K_y is what makes the second condition work, as the derivation shows.

For the second condition we need the rate of V. By the chain rule (G-371), and since the rate of a cosine is minus the sine, the two parts change at:

$$\frac{d}{dt}\frac{x_e^2}{2} = x_e \dot{x}_e$$

$$\frac{d}{dt}(1 - \cos\theta_e) = \sin\theta_e \, \dot{\theta}_e$$

We check each step at one easy moment, the error of Figure 4c on a straight ($\omega_r = 0$): $x_e = 0.2$ m, $y_e = -0.1$ m and $\theta_e = 30°$, where $\sin\theta_e = 0.5$. There the tracker commands $v = 0.4598$ m/s and $\omega = 0.72$ rad/s, and the error rates of Section 4.2 are -0.272 m/s, 0.006 m/s and -0.72 rad/s (notebook). Each step with these numbers first, then in symbols:

$$\dot{V} = 0.2 \times (-0.272) + (-0.1) \times 0.006$$

$$+0.5 \times (-0.72)/16$$

$$\dot{V} = x_e \dot{x}_e + y_e \dot{y}_e + \sin\theta_e \, \dot{\theta}_e / K_y$$

$$-0.272 = 0.72 \times (-0.1) - 1 \times 0.2$$

$$\dot{x}_e = \omega y_e - K_x x_e$$

$$0.006 = -0.72 \times 0.2 + 0.3 \times 0.5$$

$$\dot{y}_e = -\omega x_e + v_r \sin\theta_e$$

$$-0.72 = -0.3 \times (16 \times (-0.1) + 8 \times 0.5)$$

$$\dot{\theta}_e = -v_r(K_y y_e + K_\theta \sin\theta_e)$$

$$0.2 \times (-0.272) = -0.0144 - 0.04$$

$$x_e \dot{x}_e = \omega x_e y_e - K_x x_e^2$$

$$-0.1 \times 0.006 = 0.0144 - 0.015$$

$$y_e \dot{y}_e = -\omega x_e y_e + v_r y_e \sin\theta_e$$

$$0.5 \times (-0.72)/16 = 0.015 - 0.0375$$

$$\frac{\sin\theta_e \, \dot{\theta}_e}{K_y}$$

$$= -v_r y_e \sin\theta_e - \frac{v_r K_\theta \sin^2\theta_e}{K_y}$$

$$\dot{V} = -0.04 - 0.0375 = -0.0775 \text{ m}^2/\text{s}$$

$$\dot{V} = -K_x x_e^2 - \frac{v_r K_\theta \sin^2\theta_e}{K_y}$$

The first rate uses the speed command: $v - v_r \cos\theta_e$ is $K_x x_e$. In the sum, two pairs cancel: 0.0144 against -0.0144, and 0.015 against -0.015. Each cancels for a reason we can name:

- **The turning pair**, $\omega x_e y_e$: turning only swings the reference robot round our robot's view (rule 3 of Section 4.2); it does not change the distance between them.

- **The sideways pair**, $v_r y_e \sin\theta_e$: the sideways gap grows as the heading gap carries us sideways, and the $K_y y_e$ part of the turn command, weighted by the $1/K_y$ in V, takes exactly that back. Kanayama chose the two to match.

What is left is a square times a minus sign, twice. With positive gains and a reference robot driving forward (a reference speed above zero), the rate of V is never above zero, at any error (Kanayama 1991 Prop. 1). At the start of our run it is:

$$\dot{V} = -1 \times 0.2^2 - 0 = -0.04 \text{ m}^2/\text{s}$$

The rate is zero only when $x_e = 0$ and $\sin\theta_e = 0$. Could the robot stop there with a sideways gap left? No: with y_e not zero, the heading rate $-v_r K_y y_e$ is not zero, so θ_e moves off zero at once and V falls again. So V keeps falling until the whole error is zero. The full proof that the error then reaches zero needs a reference whose forward speed stays above a positive value and whose speeds change slowly (Kanayama 1991 Prop. 2); Paden §V.B.1 states it for constant v_r and ω_r, which our timetable has on each of its three parts. A timetable that spins on the spot, with zero reference speed, breaks the condition: both feedback parts of the turn command are multiplied by v_r, so the tracker cannot correct a sideways gap there. MR §13.3.4 warns the same for its own tracker.

Figure 6 shows V on the run of Figure 5. Watch the top curve fall and the bottom curve, its rate from the formula, stay below the zero line.

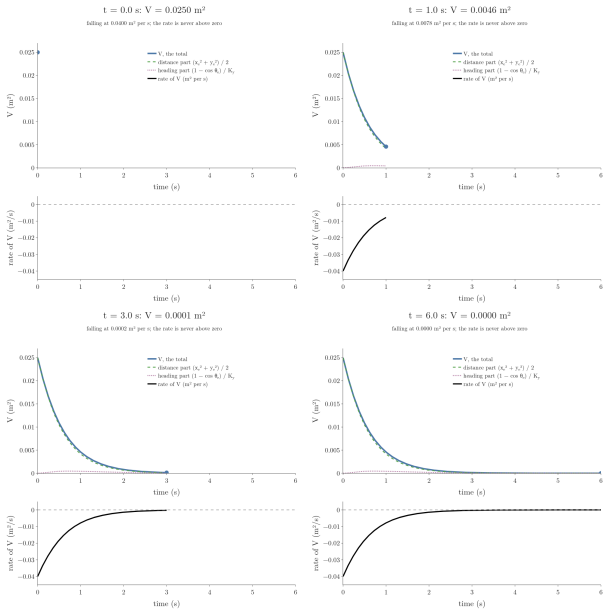


Figure 6: Kanayama's V on the run of Figure 5, one frame per 0.2 s for the first 6 s. Top: V (blue), its distance part (x_e² + y_e²)/2 (green, dashed) and its heading part (1 − cos θ_e)/K_y (purple, dotted, at most 0.0005 m²). Bottom: the rate of V from the formula −K_x x_e² − v_r K_θ sin²θ_e / K_y, with the gains K_x, K_y and K_θ and the reference speed v_r of Section 5.1. The first frame shows V = 0.025 m² falling at 0.04 m² per s; the last frame shows V near 0 and the rate never above zero

V falls from 0.025 m^2 to 0.0046 m^2 at 1 s and below 0.0001 m^2 from 3.15 s. The proof says the gap closes; it does not yet say how fast, or whether the robot swings past the line.

6.3 How fast: choosing the damping

Close to the timetable on a straight, the errors are small: $\sin\theta_e$ is about θ_e, and x_e closes on its own, by the **time constant** (G-2988) $1/K_x$, 1 s (Kanayama 1991 Prop. 3). The sideways gap and the heading gap then drive each other. A dot means the rate per second and two dots the rate of the rate, as in RO-061. With $v_r = 0.3$ m/s, $K_y = 16$ per m^2 and $K_\theta = 8$ per m, our numbers first, then the same steps in symbols:

$$\dot{y}_e = 0.3\,\theta_e$$

$$\dot{y}_e = v_r\theta_e$$

$$\dot{\theta}_e = -4.8\,y_e - 2.4\,\theta_e$$

$$\dot{\theta}_e = -v_rK_y y_e - v_rK_\theta\theta_e$$

$$\ddot{y}_e = 0.3\,\dot{\theta}_e$$

$$\ddot{y}_e = v_r\dot{\theta}_e$$

$$\ddot{y}_e = -1.44\,y_e - 2.4\,\dot{y}_e$$

$$\ddot{y}_e = -v_r^2K_y y_e - v_rK_\theta\dot{y}_e$$

$$\ddot{y}_e + 2.4\,\dot{y}_e + 1.44\,y_e = 0$$

$$\ddot{y}_e + v_rK_\theta\dot{y}_e + v_r^2K_y y_e = 0$$

The third pair used two facts: 0.3 times 4.8 is 1.44, and by the first pair 0.3 times the heading gap is the rate of the sideways gap. The last line is RO-061's second-order equation, with ω_n the **natural frequency** (G-2991), how fast the gap would swing with no damping, and ζ the **damping ratio** (G-2992), the damping compared with the damping that just stops the swing (Kanayama 1991 eq. 20):

$$\omega_n^2 = 1.44$$

$$\omega_n = 1.2 \text{ rad/s}$$

$$\omega_n = v_r\sqrt{K_y}$$

$$2\zeta\omega_n = 2.4$$

$$\zeta = 2.4/2.4 = 1$$

$$\zeta = \frac{K_\theta}{2\sqrt{K_y}}$$

A damping ratio of 1 is **critically damped** (G-2995): the fastest reply that does not swing past. Kanayama adopted it "since the convergence is fastest under non-oscillatory condition" (Kanayama 1991 §4), and so do we. Setting the damping ratio to 1 fixes the heading gain:

$$K_\theta = 2\sqrt{K_y}$$

$$= 2 \times 4 = 8 \text{ per m}$$

Figure 7 runs our start with three heading gains. Watch the orange curve cross the line and come back, the purple one creep, and the blue one arrive first, crossing by too little to see.

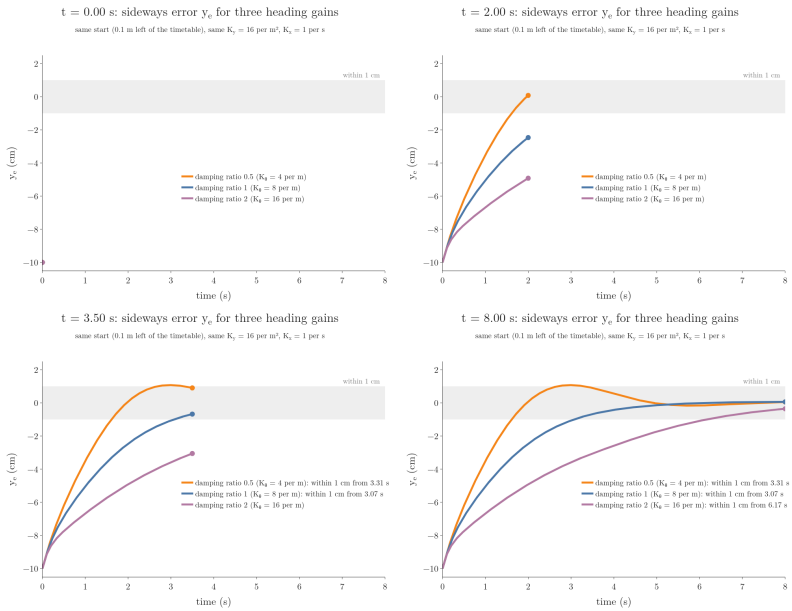


Figure 7: The sideways error y_e of Kanayama's tracker on our start for three heading gains K_θ, one frame per 0.25 s for the first 8 s; same sideways gain K_y = 16 per m^2 and speed gain K_x = 1 per s. Orange: damping ratio 0.5 (K_θ = 4 per m); blue: 1 (8 per m), this Note's choice; purple: 2 (16 per m). The grey band is ±1 cm. The last frame shows damping 0.5 within 1 cm for good from 3.31 s after crossing the line by 1.07 cm, damping 1 from 3.07 s, damping 2 from 6.17 s

Damping ratio 0.5 swings 1.07 cm past the line and settles within 1 cm from 3.31 s; damping ratio 2 creeps in and needs until 6.17 s; damping ratio 1 is within 1 cm for good from 3.07 s, crossing the line by at most 0.06 cm (notebook).

Why $K_y = 16$ per m^2. With damping ratio 1, the sideways gap at time t is its start value times a factor (Kanayama 1991 eq. 22):

$$(1 + \omega_n t)\, e^{-\omega_n t}$$

The factor is 9.2 percent when the natural frequency times the time reaches 4:

$$(1 + 4) \times e^{-4} = 0.092$$

$$t = 4/1.2 = 3.33 \text{ s}$$

$$0.3 \times 3.33 = 1.0 \text{ m}$$

In metres driven that distance is 4 divided by the square root of K_y, whatever the speed:

$$4/\sqrt{K_y} = 4/4 = 1 \text{ m}$$

So $K_y = 16$ per m^2 brings the sideways gap down to 9.2 percent within the 1 m of the first straight, before the arc starts. $K_x = 1$ per s closes the gap ahead with a 1 s time constant while keeping the first command at 0.548 m/s, under the 0.6 m/s wheel limit. The gains are tuned for small errors on a straight: far from the timetable the rule has sines and products, so its reply is not exactly this second-order one, unlike the offset point's exact 5 percent per cycle (Section 3.3). In exchange, Kanayama's tracker keeps the heading and comes with a proof.

7. Where timed tracking runs

Both trackers need a timetable from a planner and send speed and turn commands many times a second. In the stacks people use:

- **Autoware** (Figure 8). The planner hands the controller a trajectory whose points each carry a pose, a speed and a time from the start (`time_from_start` in `autoware_planning_msgs/msg/TrajectoryPoint.msg`). The trajectory follower runs every 0.03 s (`ctrl_period`) and splits the job: a lateral controller, model predictive control or pure pursuit, sets the steering, and a PID controller on the speed sets the acceleration (`autoware_trajectory_follower_node/src/controller_node.cpp`). Its `trajectory_reference_mode` is `spatial` by default, which finds the car's place on the trajectory by distance, or `temporal`, which uses each point's time as given: timed tracking (`autoware_mpc_lateral_controller/README.md`). The model predictive controller plans 50 steps of 0.1 s ahead (`lateral_controller_defaults.param.yaml`); a later Note on model predictive control for path following builds it.
- **Nav2.** Its controller plugins receive the plan as a list of poses with no times (`nav_msgs/Path` in `nav2_core/controller.hpp`) and choose the speed themselves, which is path following. The timed elastic band plans its own timed poses and turns them into commands in the same loop.
- **Kanayama's rule itself** is not one of Autoware's lateral modes (`mpc`, `pure_pursuit`) or Nav2's controller packages. Paden's survey lists it as a trajectory tracker for the kinematic model (Paden Table II), and Kanayama's group ran it on the Yamabico-11 robot (Kanayama 1991).

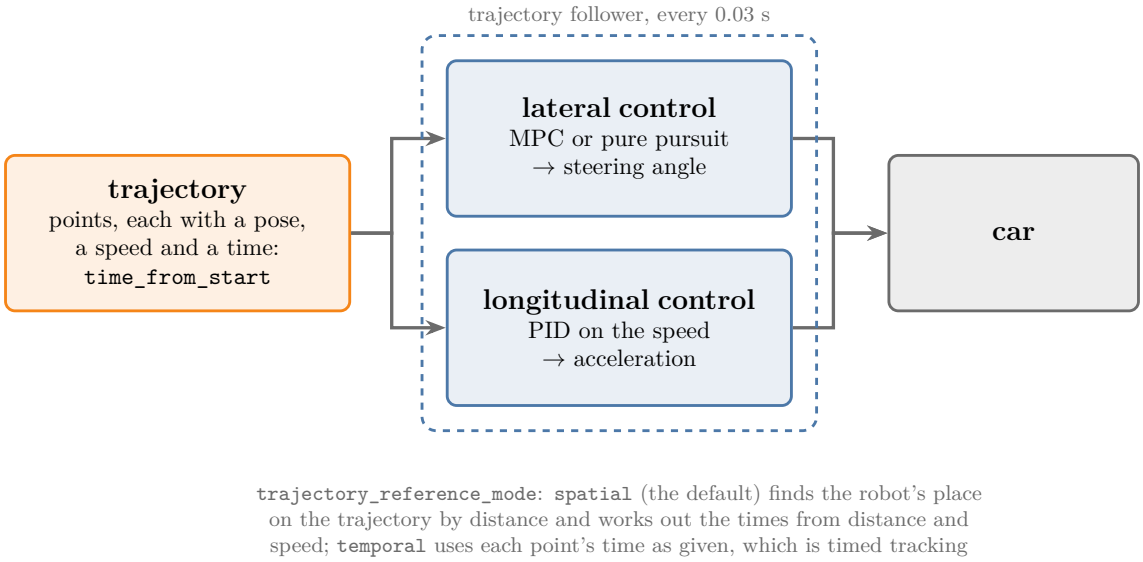


Figure 8: Autoware's trajectory follower, read from its code. The trajectory's points each carry a pose, a speed and a time (time_from_start). Every 0.03 s the lateral controller (model predictive control or pure pursuit) sets the steering angle and the longitudinal controller (PID on the speed) sets the acceleration. trajectory_reference_mode spatial, the default, places the car on the trajectory by distance; temporal uses the points' times, which is timed tracking

8. Summary

Tracker	What it uses	On our start	Why
Replay the timetable's speeds	v_r, ω_r only	0.219 m off at 11.90 s	nothing measures the gap
Path follower	the nearest path point	on the line, 0.69 s late	it ignores the clock
Offset point, P ahead	P's gap, feedforward plus k times the gap	P's gap 0.080 m at 1 s, on the timetable at the end	the gap shrinks 5 percent per cycle
Offset point, P behind	the same law	P on time, body backwards, axle 0.40 m off	the heading is not in the law
Kanayama's tracker	feedforward plus feedback on the error in the robot's frame	within 1 cm from 3.35 s	its V can only fall

- The opening's robot, 0.224 m off its timetable, is within 1 cm of it from 3.35 s and 0.0001 m off at 11.90 s under Kanayama's tracker, where replaying the speeds stayed 0.219 m off and a path follower arrived 0.69 s late, because the tracker measures the gap and closes it while feedforward keeps pace with the timetable.
- The **reference robot** (G-3180) drives the timetable exactly, so tracking a timed trajectory becomes staying on top of one moving robot; replaying its speeds alone keeps any start gap for ever, because two robots facing the same way with the same commands move alike.
- **Feedback linearisation** (G-3184) of an **offset point** (G-3185) $d = 0.2$ m ahead makes P's gap shrink 5 percent every 0.05 s at any size, because P moves ahead at the robot's speed and sideways at d times the turn rate, so it can move any way; with P behind the axle the same law keeps P on time but drives the robot backwards, because the heading is not in the law.
- The **error in the robot's frame** (G-3181), x_e ahead, y_e to the left and θ_e in heading, says what to do, because the robot's speed acts along x_e and its turn on all three; the same 0.2 m and 0.1 m gap in the room is 0.071 m ahead and 0.212 m to the right for a robot facing 45°.
- **Kanayama's tracker** (G-3182) sends the reference robot's speed and turn rate plus feedback on the three errors, so it needs no gap to keep pace; without the feedforward parts the robot sits 0.3 m behind, because then the speed must come from the gap.
- Its **Lyapunov function** (G-3183) V, 0.025 m^2 at our start, is zero only on the timetable and has a rate never above zero, so the error cannot run away from any start while the reference robot drives forward, and V keeps falling until the error is zero.
- A damping ratio of 1 (heading gain 8 per m with sideways gain 16 per m^2) closes the sideways gap fastest without crossing the line (within 1 cm from 3.07 s, against 3.31 s for 0.5 and 6.17 s for 2), and that sideways gain leaves 9.2 percent of the gap after 1 m, so the gap is gone before the arc.

The room robot can now keep to a clock, not just to a line, with a guarantee that the gap shrinks. Still missing for the chapter's car: its three path trackers have not been compared on one course with one set of numbers, and on the arc a tracker that steers only from its errors sits off the line until it is told the path's curvature, which is the next Note, feedforward on curves and choosing a tracker.

9. Sources

Built from

- Northwestern Robotics, "Modern Robotics, Chapter 13.3.4: Feedback Control for Nonholonomic Mobile Robots", YouTube, https://www.youtube.com/watch?v=STF1Ubf3moU (transcript on the book's page, https://modernrobotics.northwestern.edu/nu-gm-book-resource/13-3-4-feedback-control-for-nonholonomic-mobile-robots/)
- NPTEL-NOC IITM, "Kinematic control of Land-based Mobile Robots" (Wheeled Mobile Robots, mod07lec36), YouTube, https://www.youtube.com/watch?v=W7Fca8ZWNfA (NPTEL L36)
- Kanayama, Y., Kimura, Y., Miyazaki, F. and Noguchi, T. (1991). "A Stable Tracking Control Method for a Non-Holonomic Mobile Robot". *IEEE/RSJ International Workshop on Intelligent Robots and Systems (IROS '91)*: §1 (why a Lyapunov function), §2.2 eq. 3 (error), Lemma 1 (error rates), eq. 8 (the rule), Prop. 1 and 2 (Lyapunov function and convergence), Prop. 3, eq. 20 to 22 and §4 (damping). Free: https://hdl.handle.net/10945/40159 (Kanayama 1991)
- Paden, B., Čáp, M., Yong, S. Z., Yershov, D. and Frazzoli, E. (2016). "A Survey of Motion Planning and Control Techniques for Self-driving Urban Vehicles". *IEEE Transactions on Intelligent Vehicles* 1(1): §IV.B (trajectory planning), §V.B.1 (control Lyapunov design), §V.B.2 (output feedback linearization), Table II. Free: https://arxiv.org/abs/1604.07446 (Paden)
- Lynch, K. M. and Park, F. C. (2017). *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press. §11.3.1 eq. 11.11 to 11.15 (feedforward, proportional feedback and their sum), §13.3.1 eq. 13.19 to 13.21 and Figure 13.12 (a point on the chassis), §13.3.4 eq. 13.29 to 13.31 and Figure 13.19 (feedback control). Free preprint: http://modernrobotics.org (MR)

Other references

- Autoware Universe, https://github.com/autowarefoundation/autoware_universe, read 2026-10-12: `controller_node.cpp` and `trajectory_follower_node.param.yaml` in `control/autoware_trajectory_follower_node`; `README.md` and `lateral_controller_defaults.` in `control/autoware_mpc_lateral_controller`
- Autoware messages, https://github.com/autowarefoundation/autoware_msgs, read 2026-10-12: `TrajectoryPoint.msg` in `autoware_planning_msgs`
- Nav2, https://github.com/ros-navigation/navigation2, read 2026-10-12: `controller.hpp` in `nav2_core`; `nav2_params.yaml` in `nav2_bringup` (`controller_frequency: 20.0`)

10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Reference robot (G-3180)	An imagined robot that drives a timed trajectory exactly, so at every moment its pose and its speeds are what the real robot's should be; tracking a timetable then means staying on top of this one moving robot.
Offset point (G-3185)	A point fixed to a wheeled robot a distance d off its axle line, such as 0.2 m ahead of the axle midpoint; it moves ahead at the robot's speed and sideways at d times the turn rate, so unlike the axle midpoint it can be steered in any direction.
Feedback linearisation (G-3184)	Choosing the commands through the inverse of a robot's model so that a chosen output obeys a linear rule exactly, such as a point's gap to its reference shrinking at k times itself; it makes the error's reply the same at every size, so one gain sets it.
Error in the robot's frame (error configuration) (G-3181)	The gap from a robot to its reference measured along the robot's own axes: x_e ahead, y_e to the left and θ_e the reference heading minus the robot's; it tells the robot directly whether to speed up or turn, because its speed acts along x_e and its turn on all three.
Kanayama's tracker (G-3182)	A trajectory tracker for a wheeled robot that sends the reference's speed and turn rate plus feedback on the error in the robot's frame, $v = v_r \cos\theta_e + K_x x_e$ and $\omega = \omega_r + v_r(K_y y_e + K_\theta \sin\theta_e)$; a Lyapunov function proves its error cannot run away while the reference drives forward.
Lyapunov function (G-3183)	A number built from a system's error that is zero only at zero error, positive otherwise, and never rises while the system runs, like the height of a marble in a bowl with friction; finding one proves the error cannot run away, without trying every start.

Term	Meaning
Local planner (G-2911)	The planner that turns the next few metres or seconds of the global plan into a timed trajectory within the robot's limits and around obstacles its sensors see, solving again many times a second, such as TEB at 10 Hz over 1.5 m; it handles what the global plan cannot.
Trajectory (G-2886)	The state of a robot at every moment of a time interval, together with the controls that produce it, such as a local planner's band of poses with times; unlike a path, it says when the robot is where and how fast it moves.
Pure pursuit (G-3150)	A path tracker that, at every tick, finds the point on the path one look-ahead distance from the rear axle and steers along the circle that leaves the car along its heading and passes through that point, of curvature $2y/l_d^2$; looking ahead brings the heading error into the steering, so a car whose steering answers at once returns to the path with little swing past it.
Stanley controller (front-wheel feedback) (G-3160)	A path tracker that measures the lateral and heading errors at the front axle and steers the front wheels parallel to the path and then towards it by $\tan^{-1}(ke/(k_s + v))$, aiming them at a path point ahead; the front-axle error then shrinks by the same fraction each second, but the law is not stable when reversing.
Path following (G-3130)	Keeping a vehicle close to a path, a curve with no clock, at a speed of its own choosing, like a car on a road; a delay only makes it arrive later, so it needs only the lateral and heading errors.
Trajectory tracking (G-3131)	Making a vehicle be at each point of a timed trajectory at its time, like a train meeting its signals; a delay leaves an along-track error, which the vehicle must make up by changing speed.

Term	Meaning
Tracking (G-2954)	Making an output follow a reference that changes over time, such as a planned speed rising at 0.25 m/s^2; also called the servo problem, it needs the controller to keep up with the change, and a correction from the error alone lags behind.
Reference (G-2944)	The value we want the controlled output to have, such as a wanted speed of 0.5 m/s from the plan; the controller compares the measured output with it.
Turning radius R (G-2293)	The distance from the point a robot is turning about (the instantaneous centre of curvature, ICC) to the robot's reference point; $R = v/\omega$, so it says how tight a turn is.
Kinematic model (G-2297)	Equations that give how a robot's configuration changes from its speed inputs alone, ignoring masses and forces, such as $\dot{x} = v\cos\theta$, $\dot{y} = v\sin\theta$, $\dot{\theta} = \omega$; used to predict motion at low speed.
Feedforward (G-3070)	A control force computed from the plan and the robot's model, such as mass times the planned acceleration plus drag ($ma_r + bv_r$), and sent before any error appears; it lets the robot follow a changing plan without first falling behind, while feedback removes what the model gets wrong.
Nonholonomic constraint (G-2302)	A rule on velocities that does not follow from any equation on the pose, such as a wheel that cannot slide sideways; it limits which way a robot can move at each moment but not which poses it can reach.
Linear function (G-2527)	A function $y = ax + b$: a fixed multiple of the input plus a constant, whose graph is a straight line; passing a normal variable through it gives another normal variable, which is why the Kalman filter is exact for linear models.

Term	Meaning
Along-track error (G-3137)	How far behind (or ahead of) its timed reference a vehicle is, measured along the path, such as 3.0 m after a 1.5 s hold against a 2 m/s timetable; trajectory tracking drives it to zero by changing speed, path following ignores it.
Lateral error e (cross-track error) (G-3135)	The signed distance from a path to a point of the vehicle, measured along the path's normal at the nearest path point and positive to the left, such as -0.770 m for a car outside an arc; it is one of the two numbers a path tracker drives to zero.
Heading error θ_e (G-3136)	A vehicle's heading minus the path's direction at the nearest path point, wrapped into $-180°$ to $180°$, such as $10° - 21.8° = -11.8°$; it says whether the vehicle is turning back to the path or away, and the lateral error changes at $v \sin \theta_e$.
Error dynamics (G-2986)	The differential equation the error of a controlled system obeys, found by putting the controller's rule into the plant's equation of motion, such as 0.1 e" + 0.08 e' + 0.4 e = 0 for the heading under PD control; its solution predicts the step response from the gains.
Proportional control (P control) (G-2958)	Feedback that sets the command in proportion to the control error, such as a torque of 0.2 N m per rad of heading error; it pulls the output towards the reference like a spring, but alone it swings when nothing damps the motion and leaves an error under a steady load.
Chain rule (G-371)	To differentiate a function of a function, multiply the outer derivative by the inner derivative.
Time constant (G-2988)	The time a first-order system takes to close 63 % of the gap to its final value, m/(K_v + b) = 1 s for the robot's speed loop; it fixes the rise time (about 2.2 time constants) and the 2 % settling time (about 4).

Term	Meaning
Natural frequency (G-2991)	The rate, in rad/s, at which a second-order system would swing with no damping, $\omega_n = \sqrt{K_p/I}$, 2 rad/s for the heading loop; it sets how fast the reply is, and raising it squeezes the whole reply in time.
Damping ratio (G-2992)	A second-order system's damping compared with the damping that just stops it from swinging, $\zeta = K_d/(2\sqrt{K_p I})$, 0.2 for D gain 0.08 and 1 for 0.4; it alone sets the shape of the reply and its overshoot.
Critically damped (G-2995)	A second-order system with damping ratio exactly 1: its characteristic equation has one double real root, so it gives the fastest step response that does not overshoot, such as the turn with K_p = K_d = 0.4 settling in 2.92 s.